(1) Pipeling:

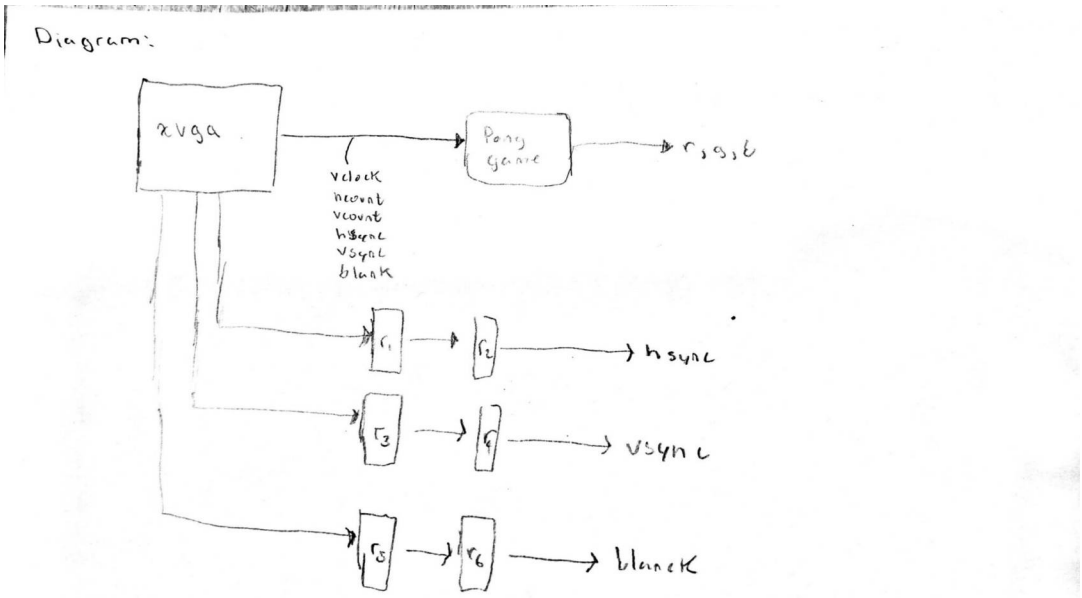
```
logic [$size(hcount) - 1: 0] old_delta_x; // can't be bigger than the screen
logic [$size(vcount) - 1: 0] old_delta_y; // can't be bigger than the screen
logic [2*($size(hcount) - 1): 0] delta_x_sq; // times 2 for squaring
logic [2*($size(hcount) - 1): 0] delta_y_sq; // times 2 for squaring

always_comb begin // generate round puck
// compute x-xcenter and y-ycenter
radiussquared = RADIUS*RADIUS; // RADIUS is a paramater
deltax = (hcount > (x+RADIUS)) ? (hcount-(x+RADIUS)) : ((x+RADIUS)-hcount);
deltay = (vcount > (y+RADIUS)) ? (vcount-(y+RADIUS)) : ((y+RADIUS)-vcount);
// check if distance is less than radius squared
if(delta_x_sq + delta_y_sq <= radiussquared) begin
      pixel_out = COLOR;
end else begin
      pixel_out = 0;
end
always_ff @(posedge clk_in) begin
    // Basically Seperated out the combinational logic for the square between
    // two registers
    //four registers two for x and 2 for y
    deltax <= old_delta_x;
    deltay <= old_delta_y;
    delta_x_sq <= old_delta_x*old_delta_x;
    delta_y_sq <= old_delta_y*old_delta_y;
end
```

(1.5) Explanation:

I decided to go with a register followed by the combinational logic followed by another register, for each delta x and delta y. This effectively isolates my multlication logic and keeps it ready for the cicle combinational logic. Lowering the progogation delay for the pong game module. This however makes my hsync, vsync, and blank run two clock cycles ahead of pong game. So we need to add tw0 registers to each wire to keep the signals in sync, and avoid pipeling issues. This is demonstrated in the diagram below.

(2) Diagram:



(3) FULL TMDS:

```
`default_nettype none
module tmds_transmitter(input wire clk_in,
```

```
                        input wire rst_in,
                        input wire [7:0] data_in,
                        output logic[9:0] data_out);
      logic [8:0] s1_to_s2;
      stage_1 my_stage_1(.clk_in(clk_in),.rst_in(rst_in),.s1_in(data_in), .s1_out(s1_t
o_s2));
      stage_2 my_stage_2(.clk_in(clk_in),.rst_in(rst_in),.s2_in(s1_to_s2),.s2_out(data
_out));
endmodule

////////////////////////////////////////////////////////////////////////////////
/// MODIFICATIONS START HERE
////////////////////////////////////////////////////////////////////////////////

module stage_1 (input wire clk_in, input wire rst_in, input wire [7:0] s1_in, output l
ogic [8:0] s1_out);
        //STAGE 1 OF TDMS
     logic [7:0] op1, op2, transitions1, transitions2;
     logic [3:0] tally1, tally2;
     logic [9:0] out;
     //generates options
     assign op1 = {s1_in[7] ^ s1_in[6], s1_in[6] ^ s1_in[5], s1_in[5] ^ s1_in[4], s1_i
n[4] ^ s1_in[3], s1_in[3] ^ s1_in[2], s1_in[2] ^ s1_in[1], s1_in[1] ^ s1_in[0], s1_in[
0]};
     assign op2 = {!(s1_in[7] ^ s1_in[6]), !(s1_in[6] ^ s1_in[5]), !(s1_in[5] ^ s1_in[
4]), !(s1_in[4] ^ s1_in[3]), !(s1_in[3] ^ s1_in[2]), !(s1_in[2] ^ s1_in[1]), !(s1_in[1
] ^ s1_in[0]), s1_in[0]};
     //gets transitions
     assign transitions1 = {(op1[7]^op1[6]),(op1[6]^op1[5]), (op1[5]^op1[4]),(op1[4]^o
p1[3]),(op1[3]^op1[2]),(op1[2]^op1[1]),(op1[1]^op1[0]), op1[0]};
     assign transitions2 = {(op2[7]^op2[6]),(op2[6]^op2[5]), (op1[5]^op2[4]),(op2[4]^o
p2[3]),(op2[3]^op2[2]),(op2[2]^op2[1]),(op2[1]^op2[0]), op2[0]};
     //tally counter instantiations
     tallier t1(.byte_in(transitions1), .tally_out(tally1));
     tallier t2(.byte_in(transitions2), .tally_out(tally2));
     assign out = (tally1 < tally2) ? {1'b1, op1} : {1'b0, op2};

     always_ff @(posedge clk_in) begin
        if(rst_in) begin
            s1_out <= 9'b0_0000_0000;
        end else begin
            s1_out <= out;
        end
     end
endmodule

module tallier(
        // counts number of ones in a byte
    input wire [7:0] byte_in,
    output logic [3:0] tally_out
    );
    assign tally_out = byte_in[7] + byte_in[6] + byte_in[5] + byte_in[4] + byte_in[3]
+ byte_in[2] + byte_in[1] + byte_in[0];
endmodule

module stage_2 (input wire clk_in, input wire rst_in, input wire [8:0] s2_in, output l
ogic[9:0] s2_out);
        //STAGE 2 OF TDMS
    logic major_in, major_out;
    majority maj_in(.data_in(s2_in), .majority_out(major_in));
    majority maj_out(.data_in(s2_out[8:0]), .majority_out(major_out));
    always_ff @(posedge clk_in ) begin
        if (rst_in) begin
                s2_out <= 0;
        end else begin
```

```
            if(major_in) begin // check current
                //checks previous before mapping it current to output
                s2_out <= (major_out) ? {1'b1, ~s2_in[8:0]} : {1'b0, s2_in[8:0]};
            end else begin
                s2_out <= (major_out) ? {1'b0, s2_in[8:0]} : {1'b1, ~s2_in[8:0]};
            end
        end
    end
endmodule

module majority (
        //this module determines if a 9-bit is made up of mostly ones or
        //zeros. if it has mostly ones majority_out pulls high else
        //low.
    input wire [8:0] data_in,
    output logic majority_out
);
    logic [3:0] temp;
    assign temp = (data_in[8] + data_in[7] + data_in[6] + data_in[5]+ data_in[4]+ data
_in[3]+ data_in[2]+ data_in[1] + data_in[0]);
    assign majority_out = (temp > 4);
endmodule

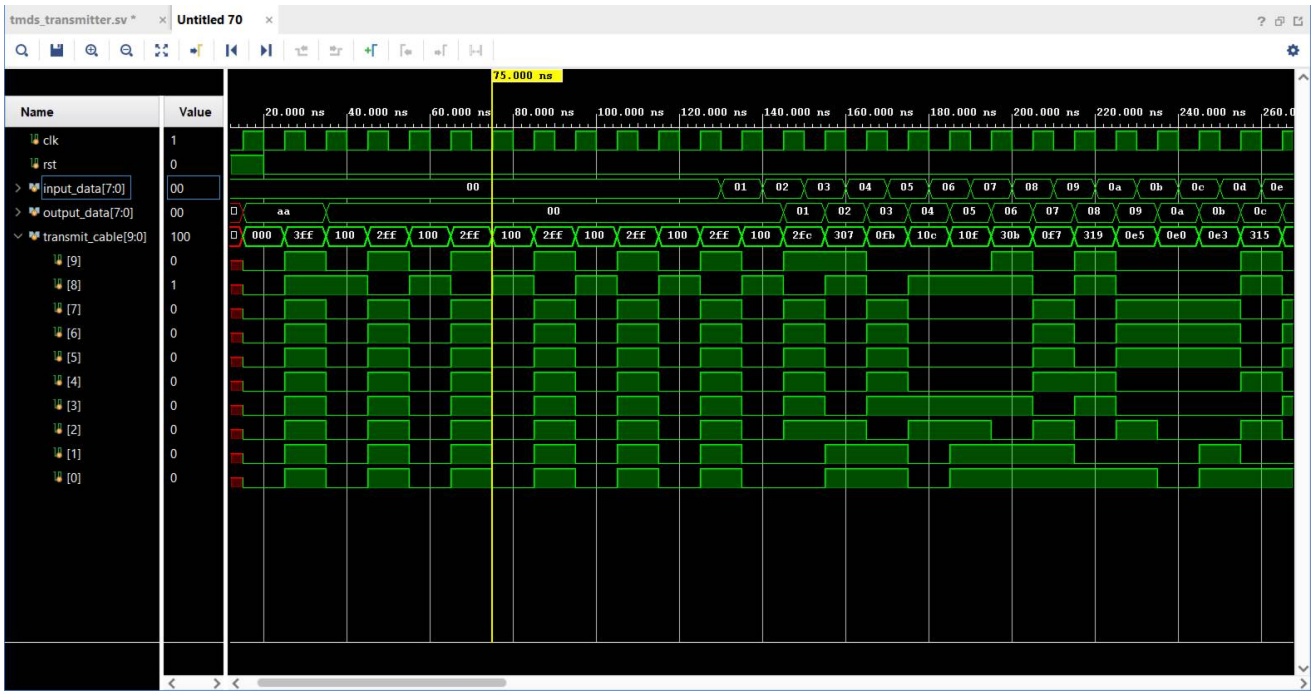
////////////////////////////////////////////////////////////////////////////////
/// MODIFICATIONS END HERE
////////////////////////////////////////////////////////////////////////////////

//provided for your usage:
module tmds_receiver (input wire [9:0] data_in, output logic[7:0] data_out);
        logic[8:0] stage_1;
        assign stage_1 = data_in[9]? ~data_in[8:0]:data_in[8:0];
        logic [7:0] option_1;
        logic [7:0] option_2;
        xor_decode my_xord   (.data_in(stage_1[7:0]),.data_out(option_1));
        xnor_decode my_xnord (.data_in(stage_1[7:0]),.data_out(option_2));
        assign data_out = stage_1[8]?option_1:option_2;
endmodule

module xor_decode (input wire [7:0] data_in, output logic[7:0] data_out);
    always_comb begin
        data_out = 8'b0; //initialize
        data_out[0] = data_in[0];
        for(integer i=1; i<8; i++)begin
            data_out[i] = data_out[i-1]^data_in[i];
        end
    end
endmodule

module xnor_decode (input wire [7:0] data_in, output logic[7:0] data_out);
    always_comb begin
        data_out = 8'b0; //initialize
        data_out[0] = data_in[0];
        for(integer i=1; i<8; i++)begin
            data_out[i] = ~(data_out[i-1]^data_in[i]);
        end
    end
endmodule
`default_nettype wire
```

(5) Testbench:


tmds_transmitter.sv *
Untitled 70
Name
Value
clk
1
rst
0
input_data[7:0]
00
output_data[7:0]
00
transmit_cable[9:0]
100
[9]
0
[8]
1
[7]
0
[6]
0
[5]
0
[4]
0
[3]
0
[2]
0
[1]
0
[0]
0
75.000 ns
20.000 ns
40.000 ns
60.000 ns
80.000 ns
100.000 ns
120.000 ns
140.000 ns
160.000 ns
180.000 ns
200.000 ns
220.000 ns
240.000 ns
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e
aa 00 01 02 03 04 05 06 07 08 09 0a 0b 0c
000 3ff 100 2ff 100 2ff 100 2ff 100 2ff 100 2ff 100 2fc 307 0fb 10c 10f 30b 0f7 319 0e5 0e0 0e3 315